OPERATING SYSTEMS AND COMPUTER NETWORKS

SYSTEM CALLS AND STANDARD C LIBRARY

Lecturer: Milos Antonijevic

SYSTEM CALLS

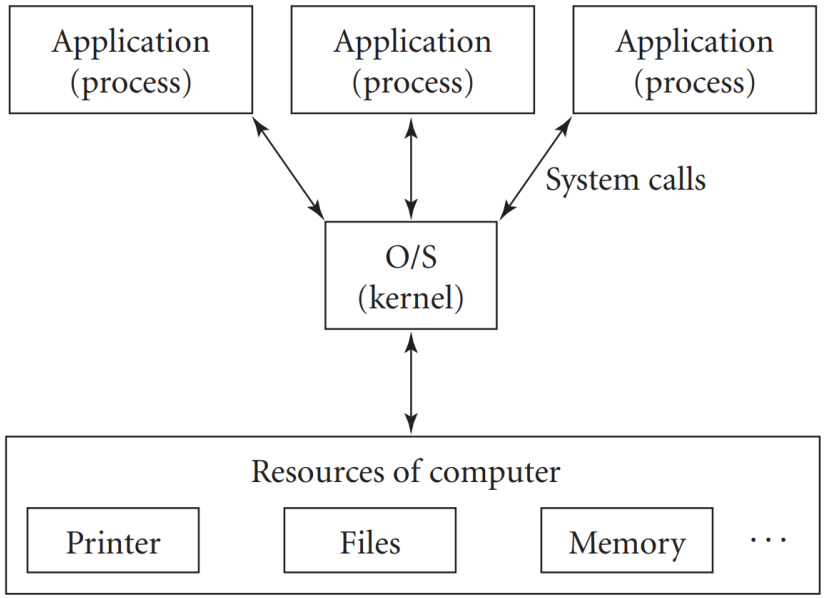
DEFINITION:

System calls are function invocations made from user space (user application) into the kernel (the core internals of the system) in order to request some service or resource from the operating system.

- In the Linux kernel, each machine architecture implements its own list of available system calls. However, a very large subset of system calls (more than 90%) is implemented by all architectures.
- It is not possible to directly link user-space applications with kernel space. For reasons of security and reliability, user-space applications must not be allowed to directly execute kernel code or manipulate kernel data. Instead, the kernel must provide a mechanism by which a user-space application can "signal" the kernel that it wishes to invoke a system call.

SYSTEM CALLS

- Linux kernel provides several hundred different system calls. These system calls are typically broken up into families of functions.
- **Memory management** system calls ask the O/S to manipulate a block of memory in some manner, typically so that the memory can be used by an application program. These operations involve manipulating the low-level attributes of memory as managed by the operating system.
- **Time management** system calls ask the OS to access the system clock. These system calls either retrieve values from the system clock, or start or stop timers based upon the system clock.
- **File management** system calls ask the OS to access a file or device.
- **Process management** system calls ask the O/S to run another program, or control how it runs.
- **Signal management** system calls provide a rudimentary form of interprocess communication.
- **Socket management** system calls allow a program on one computer to communicate with a program on a second computer through a network.



SYSTEM CALLS

- Many standard library functions are built on top of system calls, in other words the library functions use system calls as part of their code.
- malloc() family of functions is built on top of the mmap() and brk() functions. The malloc() function is in the C standard library, while mmap() is a system call.
- sleep() library function is implemented by calling a few time management system calls, such as alarm(). The sleep() function provides a single function to pause a program for a specified amount of time.
- fopen(), fread(), fwrite() and fclose() library functions are build on top of the open(), read(), write(), and close() system calls.
- Library functions usually provide a simpler interface to application programs and are generally more portable than system calls (since system calls provide direct access to the kernel, different operating systems will have at least slightly different system calls).
- Library functions sometimes provide additional capabilities not available directly through system calls. For example, the C standard library functions fopen(), fread(), fwrite(), and fclose() provide buffering, whereas the system calls open(), read(), write(), and close() do not.

FILES

- Before a file can be read from or written to, it must be opened.
- The kernel maintains a per process list of open files, called the file table. This table is indexed via nonnegative integers known as **file descriptors**.
- Each entry in the list contains information about an open file, including a pointer to an in-memory copy of the file's backing inode and associated metadata.
- Opening a file returns a file descriptor, while subsequent operations (reading, writing, and so on) take the file descriptor as their primary argument.
- Each Linux process has a maximum number of files that it may open. File descriptors start at 0.
- Every process by convention has at least three file descriptors open: File descriptor 0 is standard in (stdin), file descriptor 1 is standard out (stdout), and file descriptor 2 is standard error (stderr). [Remember redirecting in Linux, for example, with `find /etc -name 'passwd' 2> error.txt 1> output.txt`]

FILES

- Standard C I/O library functions do not operate directly on file descriptors. Instead, they use their own unique identifier, known as the **file pointer**. Inside the C library, the file pointer maps to a file descriptor.
- In I/O context, an open file is called a stream. Streams may be opened for reading (input streams), writing (output streams), or both (input/output streams).
- Creating (or opening) a file can be done directly by using system call open():

```
int fd = open("example.txt", O_CREAT|O_WRONLY|O_TRUNC);
//or with
//int fd = creat("foo");
```

- or by using C library function fopen():

```
FILE fp = fopen("example.txt", "w");
```

FILES

- We can follow system calls made by the program by using Linux **strace** tool.

```
student@linux$ echo 'example text' > file.txt
student@linux$ strace cat file.txt
```

- From the output we can see that **cat** open a file for reading and that it return a file descriptor with the value of 3 (because every process already have 0-2 descriptors).
- read() is used to read the bytes from a file (identified by file descriptor in the argument).
- write() is used to display the content of the file to the stdin (1).

OPENING A FILE

- For opening a file, we can use **fopen()** function. The first argument is a file pointer and the second is mode that describes how to open a file.

File Mode	Description
r	Open a file for reading only.
w	Open a file for writing. A new file is created if doesn't exist. If a file exists on a system, then all the data inside the file is truncated to zero length.
a	Open a file in append mode. File is created if doesn't exist. The content within the file doesn't change.
r+	Open for reading and writing from beginning.
w+	Open for reading and writing, overwriting a file.
a+	Open for reading and writing, appending to file.

WRITING TO FILE

- stdio C library offers the 3 functions to write to a file:
  - **fputc(char, fp)** - writes a character to the file pointed to by fp.
  - **fputs(str, fp)** - writes a string to the file pointed to by fp.
  - **fprintf(fp, str, variableLists)** - writes a string to the file pointed to by fp with format specifiers and a list of variables variableLists.

```
#include <stdio.h>
int main() {
    int i;
    FILE * fp;
    char str[] = "It is working\n";
    fp = fopen("test1.txt", "w");
    for (i = 0; str[i] != '\n'; i++)
        fputc(str[i], fptr);
    fclose(fptr);
    return 0;
}
```

```
#include <stdio.h>
int main() {
    FILE * fp;
    fp = fopen("test2.txt", "w+");
    fputs("Simplier than fputc() function\n", fp);
    fclose(fp);
    return (0);
}
```

READING FROM A FILE

- stdio C library offers the 3 functions to read from a file:
  - **fgetc(fp)** - returns the next character from the file pointed to by the file pointer. When the end of the file has been reached, the EOF is sent back.
  - **fgets(buffer, n, fp)** - reads n-1 characters from the file and stores the string in a buffer in which the NULL character '\0' is appended as the last character.
  - **fscanf(fp, conversionSpecifiers, variables)** - reads a word from the file and assigns the input to a list of variables using conversion specifiers. fscanf() stops reading a string when space or newline is encountered.

```
#include <stdio.h>

main() {
   FILE *fp;
   char buff[255];
   fp = fopen("file.txt", "r");
   while(fscanf(fp, "%s", buff)!=EOF) printf("%s ", buff );
   fclose(fp);
}
```

MOVING INSIDE A FILE

- stdio C library offers a function (based on lseek() system call) to manipulate the current stream position. The function is **fseek()**.
- If last argument is set to SEEK_SET, the file position is set to offset. If last argument is set to SEEK_CUR, the file position is set to the current position plus offset. If last argument is set to SEEK_END, the file position is set to the end of the file plus offset.

```
#include <stdio.h>
void main(){
   FILE *fp;
   fp = fopen("file.txt","w+");
   fputs("This is system programing", fp);
   fseek(fp, 7, SEEK_SET);
   fputs("something else", fp);
   fclose(fp);
}
```

USING fsync() AND fflush() FUNCTIONS

- In order understand the effect of **fflush()**, we have to understand the difference between the buffer maintained by the C library, and the kernel's own buffering.
- Buffer that is maintained by the C library resides in user space, for the performance purposes. A system call is issued only when the disk or some other medium has to be accessed.
- The **fflush()** function writes out the user buffer to the kernel, ensuring that all data written to a stream is flushed via write() system call.
- However, this does not guarantee that the data is physically committed to any medium. Most of the time when a program calls write(), it is just telling the file system to write the data to persistent storage, at some point in the future. This means that if OS crashes after the write() call and before the actual writing to the disk, the data will be lost.
- The **fsync()** function forces all not written data for specified file descriptor to be committed to disk.

READING AND SETTING FILE PERMISSIONS

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/stat.h>

void output_permissions(mode_t m) {
    putchar( m & S_IRUSR ? 'r' : '-' );
    putchar( m & S_IWUSR ? 'w' : '-' );
    putchar( m & S_IXUSR ? 'x' : '-' );
    putchar( m & S_IRGRP ? 'r' : '-' );
    putchar( m & S_IWGRP ? 'w' : '-' );
    putchar( m & S_IXGRP ? 'x' : '-' );
    putchar( m & S_IROTH ? 'r' : '-' );
    putchar( m & S_IWOTH ? 'w' : '-' );
    putchar( m & S_IXOTH ? 'x' : '-' );
    putchar('\n');
}

int main(int argc, char *argv[]) {
    const char *filename;
    struct stat fs;
    int r;

    filename = "test.file";
    printf("Permissions for '%s':\n",filename);
    r = stat(filename,&fs);
    puts("Current permissions:");
    output_permissions(fs.st_mode);
    puts("Set group and other write permissions");


    r = chmod( filename, fs.st_mode | S_IWGRP+S_IWOTH );
    if( r!=0) {
        fprintf(stderr,"Unable to reset permissions on '%s'\n",filename);
        exit(1);
    }
    puts("Updated permissions:");
    stat(filename,&fs);
    output_permissions(fs.st_mode);


    return(0);
}
```

https://www.gnu.org/software/libc/manual/html_node/Permission-Bits.html

SETTING EFFECTIVE UID OF THE PROCCESS

- When the **setuid** bit is used, the file is not executed with the privileges of the user who launched it, but with the privileges of the file owner instead.

```
student@linux# chown root a.out
student@linux# chmod u+s a.out
```

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main() {
        setuid(0);
        system("/bin/bash");
}
```